Assignment A7 — FL Robustness

CS-E4740 Federated Learning

Read Carefully

There is no submission. Your work is assessed via a multiple-choice quiz.

You must compute and print the numerical quantities specified in this document.

The assessed quantities are:

1. **Federated linear regression results**
   - average validation MSE within each cluster for each of the three graph variants.

No other quantities are graded. Plots are optional and are not assessed directly.

Python Environment

To ensure reproducibility, the following environment is recommended:

- Python $\geq$ 3.11
- `numpy` == 2.3.4
- `scikit-learn` == 1.7.2 (for `LinearRegression`)
- `networkx` == 3.5 (for representing the node set)
- `matplotlib` == 3.10.7 (optional; plotting not required)

You can verify installed package versions via:

```
python -m pip show numpy scikit-learn networkx matplotlib
```

1 Purpose

This assignment studies the robustness of a federated learning system in the presence of heterogeneous clusters. Using daily weather observations from Finnish Meteorological Institute (FMI) stations, each node first trains a local linear model. The resulting local learning problems are summarized by deterministic station-specific representation vectors. These representations are then used to partition the nodes into clusters with different statistical characteristics.

By constructing clustered communication graphs and varying the number of inter-cluster edges, the assignment examines how graph structure influences graph-coupled learning. In particular, it studies the trade-off between preserving cluster-specific local structure and enabling information sharing across heterogeneous groups of nodes.

2 Data

We use the same raw data as in the previous assignment. In particular, you must use the following CSV file (authoritative source):

https://github.com/alexjungaalto/FederatedLearning/blob/main/Edition2026/assignments/fmidata.csv

Each row corresponds to **a specific day at a specific FMI weather station**. The CSV file contains the following columns:

`station`	station name (string)
`lat`	latitude (float)
`lon`	longitude (float)
`day`	date in `YYYY-MM-DD` format
`tmax`	daily maximum temperature (float); used as label y
`tmin`	daily minimum temperature (float); used as feature x

The following snippet illustrates the structure of the CSV file:

```
station,lat,lon,day,tmax,tmin
Alajärvi Möksy,63.08898,24.26084,2025-12-31,-15.3,-22.0
Alajärvi Möksy,63.08898,24.26084,2026-01-01,-16.5,-22.3
Porvoo Kilpilahti satama,60.30373,25.54916,2026-01-01,-3.2,-7.8
```

All temperatures are measured in degrees Celsius (°C). **You must not modify or replace the dataset**.

3 Build a graph with one node per station and no edges yet

Construct an undirected `networkx.Graph` whose node set is

$$V = \{0, 1, \ldots, n-1\},$$

where n is the number of distinct stations in the CSV.

For each node, store:

- station name,
- latitude,
- longitude,
- observation days,
- the full local dataset.

For node i, the full local dataset must be stored as a NumPy array whose rows are ordered as

$$(\texttt{tmax}, \texttt{tmin}).$$

4 Split each local dataset into training and validation

For each node:

- the training set consists of all rows except those corresponding to the latest available day,
- the validation set consists of all rows corresponding to the latest available day.

This split must be performed independently for each node.

5 Fit one local linear-regression model per node

At node i, use the local training set to fit the model

$$y \approx w_0^{(i)} + w_1^{(i)} x,$$

where:

- $x =$ `tmin`,
- $y =$ `tmax`.

Construct the local design matrix by explicitly including the dummy feature 1:

$$X_i = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_{m_i} \end{bmatrix}, \qquad y_i = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{m_i} \end{bmatrix}.$$

You must use:

```
LinearRegression(fit_intercept=False)
```

so that the learned coefficient vector is directly

$$\widehat{w}^{(i)} = \begin{bmatrix} \widehat{w}_0^{(i)} \\ \widehat{w}_1^{(i)} \end{bmatrix}.$$

6 Cluster the nodes using the new representation

For each node i, construct the deterministic 7-dimensional representation

$$z^{(i)} = \left[\widehat{w}_1^{(i)}, \widehat{w}_0^{(i)}, \sigma_{\text{res}}^{(i)}, \overline{t_{\min}}^{(i)}, \text{std}(t_{\min})^{(i)}, \overline{t_{\max}}^{(i)}, \text{std}(t_{\max})^{(i)}\right].$$

Here:

- $\widehat{w}_1^{(i)}$ is the slope,
- $\widehat{w}_0^{(i)}$ is the intercept,
- $\sigma_{\text{res}}^{(i)}$ is the standard deviation of the local training residuals,
- $\overline{t_{\min}}^{(i)}$ and $\text{std}(t_{\min})^{(i)}$ are the mean and standard deviation of the local training values of `tmin`,
- $\overline{t_{\max}}^{(i)}$ and $\text{std}(t_{\max})^{(i)}$ are the mean and standard deviation of the local training values of `tmax`.

Then:

- standardize the representation vectors using `StandardScaler`,
- cluster them using `AgglomerativeClustering` with:
  - `n_clusters=3`,
  - `metric='euclidean'`,
  - `linkage='ward'`.

This clustering step is deterministic.

For reproducibility of the reported results, you must ensure the correct indexation of clusters as follows:

- the cluster containing **119** nodes has index **0**,
- the cluster containing **16** nodes has index **1**,
- the cluster containing **64** nodes has index **2**.

In other words, if the raw output labels produced by `AgglomerativeClustering` use a different numbering, you must apply a deterministic relabeling so that the cluster indices match the above node counts before constructing the graphs and reporting the average validation MSE values.

7 Construct edges using 3NN within each cluster

Construct the base graph as follows:

- for each cluster separately,
- connect every node to its 3 nearest neighbours within that cluster,
- nearest neighbours must be computed in the **standardized 7-dimensional representation space**.

No inter-cluster edges are present in the base graph.

8 Build three graphs with 0, 1, and 5 inter-cluster edges per cluster pair

Let $G_{\text{base}} = (V, E_{\text{base}})$ denote the base graph obtained after constructing the 3-nearest-neighbour edges within each cluster. From this base graph, construct three graph variants:

- $G_0 (= G_{\text{base}})$: the base graph itself, i.e., no inter-cluster edges are added,
- G_1: the base graph plus exactly 1 inter-cluster edge for each unordered pair of distinct clusters,
- G_5: the base graph plus exactly 5 inter-cluster edges for each unordered pair of distinct clusters.

Step 1: Consider one unordered pair of distinct clusters

Let C_a and C_b be two distinct clusters, with $a < b$. Consider all possible node pairs

$$(u, v) \in C_a \times C_b,$$

that is, all pairs such that node u belongs to cluster C_a and node v belongs to cluster C_b.

Each such pair (u, v) is a *candidate inter-cluster edge*. If this candidate is selected, then the undirected edge $\{u, v\}$ is added to the graph.

Step 2: Compute a distance for each candidate pair

For every candidate pair $(u, v) \in C_a \times C_b$, compute the squared Euclidean distance between the standardized 7-dimensional representation vectors of nodes u and v:

$$d(u, v) \triangleq \left\| \widetilde{z}^{(u)} - \widetilde{z}^{(v)} \right\|_2^2,$$

where $\widetilde{z}^{(u)}$ and $\widetilde{z}^{(v)}$ denote the standardized representation vectors.

Step 3: Sort candidate pairs in ascending order

Sort all candidate pairs (u, v) in **ascending** order according to the triple

$$\big(d(u,v),\ u,\ v\big).$$

This means:

- pairs with smaller distance are preferred,
- if two candidate pairs have exactly the same distance, the pair with smaller u is preferred,
- if both the distance and u are equal, the pair with smaller v is preferred.

Step 4: Add the required inter-cluster edges

After sorting the candidate pairs for the cluster pair (C_a, C_b):

- for G_1, add the undirected edge corresponding to the first candidate pair in the sorted list,
- for G_5, add the undirected edges corresponding to the first five candidate pairs in the sorted list.

Equivalently:

- in G_1, exactly one inter-cluster edge is added for the cluster pair (C_a, C_b),
- in G_5, exactly five inter-cluster edges are added for the cluster pair (C_a, C_b),

unless fewer than five candidate pairs exist, in which case all available candidate pairs are added.

Step 5: Repeat for every unordered pair of distinct clusters

Repeat the above procedure for every unordered pair of distinct clusters. The resulting graphs are:

- G_0: no added inter-cluster edges,
- G_1: one added inter-cluster edge per unordered pair of clusters,
- G_5: five added inter-cluster edges per unordered pair of clusters.

9 Solve GTVMin for the three graphs

For each of the three graphs, implement the following federated linear-regression method which is obtained by applying block-coordinate optimization to the GTVMin instance

$$\min_{w^{(1)},\ldots,w^{(n)}\in\mathbb{R}^2} \sum_{i\in V} L_i\Big(w^{(i)}\Big) + \alpha \sum_{\{i,j\}\in E} \Big\|w^{(i)} - w^{(j)}\Big\|_2^2$$
$$\text{with } L_i\Big(w^{(i)}\Big) = \frac{1}{|D_i^{\text{train}}|} \sum_{(x,y)\in D_i^{\text{train}}} \big(y - (w_0^{(i)} + w_1^{(i)} x)\big)^2. \tag{1}$$

In this assignment, you must use the regularization parameter

$$\alpha = 1.$$

In particular,

- For each node i, initialize $w^{(i)}$ by plain linear regression using only the local training set D_i^{train}.

- For $t = 0, 1, \ldots, 99$, perform the following synchronous updates at all nodes:

$$w^{(i,t+1)} \in \arg\min_{w \in \mathbb{R}^2} L_i(w) + \alpha \sum_{j \in N(i)} \left\| w - w^{(j,t)} \right\|_2^2. \tag{2}$$

- Store the learnt model parameters $\widehat{w}^{(i)} = w^{(i,100)}$ which are needed for computing the results in the next task.

This update can be implemented *without custom optimizers* by applying `LinearRegression.fit()` to an augmented version of the local training set D_i^{train}. You might also need to use non-uniform sample weights in `LinearRegression.fit()`.

Note:

- The update (2) must be executed *synchronously* at each node i, i.e., using neighbours' model parameters $w^{(j,t)}$, for $j \in N(i)$.
- Make sure that your Python implementation does not accidentally overwrite model parameters $w^{(j,t)}$ which are still needed on the right-hand side of the update (2) at some other nodes i.

10 Compute average validation MSE within each cluster

For each of the three graphs, and for each cluster C, compute the average validation MSE across all nodes belonging to cluster C:

$$\frac{1}{|C|} \sum_{i \in C} \frac{\|X_i^{\text{val}} \widehat{w}^{(i)} - y_i^{\text{val}}\|_2^2}{|D_i^{\text{val}}|}.$$

In words:

- first compute the validation MSE separately for each node i in cluster C,
- then average these node-wise validation MSE values over all nodes in the cluster.

11 Print results

You must print:

- average validation MSE within each cluster for G_0, G_1, and G_5.